



API DESIGN

Construyendo APIs REST

Que se entiendan, se usen, y duren en el tiempo

REST

HTTP

Diseño

Versionado

Errores

De qué vamos a hablar

Las decisiones que tomas cuando diseñas una API

1. Qué es una API REST

2. Los principios de REST

3. Recursos y URLs

4. Verbos HTTP

5. Códigos de estado

6. Diseño de respuestas

7. Manejo de errores

8. Filtros, paginación, búsqueda

9. Versionado y seguridad

10. Errores comunes

Una API mal diseñada no se rompe — pero nadie quiere usarla

1. Qué es una API REST

Una conversación entre programas usando HTTP

REST (Representational State Transfer) es un estilo de arquitectura para diseñar APIs sobre HTTP. No es un estándar ni una librería: es un conjunto de convenciones que cuando se siguen, hacen que las APIs sean predecibles.



Lo importante:

El cliente y el servidor pueden estar hechos en lenguajes distintos, en máquinas distintas, y mantenidos por equipos distintos. Solo necesitan ponerse de acuerdo en el formato de los mensajes.



2. Los principios de REST

Lo que hace que una API sea "REST" de verdad

Cliente-servidor

Separados. El cliente no sabe cómo el servidor guarda los datos; el servidor no sabe quién es el cliente.

Sin estado (stateless)

Cada petición lleva toda la info necesaria. El servidor no "recuerda" peticiones anteriores.

Recursos identificables

Todo es un recurso con URL única. /productos/42 es siempre el producto 42.

Interfaz uniforme

Mismos verbos HTTP, mismos códigos, mismas convenciones para toda la API.

Si tu API cumple los 4 → es REST. Si solo cumple algunos → es "REST-ish" (lo más común en la realidad).



3. Recursos y URLs

El sustantivo manda, no el verbo

En REST, una URL apunta a un recurso (un "sustantivo"), no a una acción. Lo que haces con él lo dice el verbo HTTP, no la URL.

Mal — verbos en la URL

```
GET /obtenerProductos
GET /listarProducto/5
POST /crearProducto
POST /actualizarProducto/5
GET /borrarProducto?id=5
```

Bien — sustantivos + verbos HTTP

```
GET /productos
GET /productos/5
POST /productos
PUT /productos/5
DELETE /productos/5
```

Plural y en minúsculas: /productos /usuarios /pedidos (no /producto, no /Productos)



4. Verbos HTTP

Cada verbo tiene un propósito y un contrato

Verbo	Para qué	¿Cambia datos?	Ejemplo
GET	Leer/consultar	No	GET /productos/5
POST	Crear un recurso nuevo	Sí (agrega)	POST /productos
PUT	Reemplazar completo	Sí (reemplaza)	PUT /productos/5
PATCH	Modificar parcialmente	Sí (parcial)	PATCH /productos/5
DELETE	Eliminar	Sí (borra)	DELETE /productos/5

Idempotencia (palabra fea, concepto útil)

Una operación es idempotente si llamarla 1 o 100 veces deja el sistema igual. GET, PUT y DELETE lo son. POST no (cada llamada crea un nuevo recurso).



5. Códigos de estado HTTP

El primer mensaje que da tu API

Antes de leer el body, el cliente mira el código de estado. Si todos respondes 200, le obligas a parsear el body para saber si funcionó.

2xx	Éxito 200 OK, 201 Created, 204 No Content
3xx	Redirección 301 Moved, 304 Not Modified
4xx	Error del cliente 400 Bad Request, 401, 403, 404
5xx	Error del servidor 500 Internal Error, 503 Unavailable

Regla práctica: si algo salió mal por culpa del cliente (datos malos, no autenticado, recurso inexistente), respondes 4xx. Si fue por culpa tuya (BD caída, bug), 5xx.

</> 6. Diseño de respuestas

Forma consistente, predecible

La forma del JSON debe ser consistente en toda tu API. Los clientes confían en eso.

Un recurso (GET /productos/5):

```
{
  "id": 5,
  "nombre": "Notebook",
  "precio": 450000
}
```

Una lista (GET /productos):

```
{
  "data": [...],
  "total": 42,
  "pagina": 1
}
```

Reglas que se pagan en cordura:

- Mismas claves en mayúscula/minúscula: **camelCase o snake_case, no ambos**
- Tipos coherentes: un **id** siempre número, no a veces string
- Listas SIEMPRE devuelven un arreglo (vacío si no hay), nunca null
- Fechas en formato ISO 8601: **"2026-05-11T14:30:00Z"**



7. Manejo de errores

Un error útil ahorra horas de debugging

Un error mal formateado te hace perder media tarde leyendo logs. Un error bien formateado se lee y se entiende.

Mal:

```
// Status: 200 OK
{
  "error": true,
  "msg": "algo fallo"
}
```

Bien:

```
// Status: 400 Bad Request
{
  "error": "validacion_fallida",
  "mensaje": "Precio invalido",
  "campo": "precio"
}
```

Errores que aman los desarrolladores:

- El status code refleja lo que pasó (400, 401, 404, 422, 500...)
- Un código de error legible que el cliente pueda chequear con un if
- Un mensaje en humano para mostrar o loggear
- Nunca devuelvas el stack trace al cliente — eso es para los logs



8. Filtros, paginación, búsqueda

Query params, no nuevas rutas

Cuando quieras filtrar, ordenar, buscar o paginar, NO inventes rutas nuevas (/productos/baratos, /productos/buscar). Usa query params sobre el mismo endpoint.

Filtrar	GET /productos?categoria=audio&disponible=true
Buscar	GET /productos?q=notebook
Ordenar	GET /productos?orden=precio_desc
Paginar	GET /productos?pagina=2&limite=20
Combinado	GET /productos?categoria=audio&orden=precio&pagina=1

En Express los lees con req.query: req.query.categoria, req.query.pagina



9. Versionado y seguridad

Dos decisiones que vas a agradecer haber tomado temprano

Versionado

Tu API va a cambiar. Si los clientes ya están usando una versión, cualquier cambio incompatible los rompe.

Estrategias comunes:

```
// En la URL (lo más común)
/api/v1/productos
/api/v2/productos

// O en un header
Accept: application/
vnd.miapi.v2+json
```

Empieza siempre con /v1 aunque no creas que vas a necesitarlo.

Seguridad

HTTPS siempre. Sin excepciones. Sin "esto es solo desarrollo".

Autenticación: típicamente con tokens (JWT, OAuth) en el header **Authorization**.

Validar siempre la entrada. Nunca confíes en lo que llega de un cliente. Tipos, longitudes, formato.

Rate limiting: limita peticiones por IP/usuario para evitar abuso.

✖ 10. Errores comunes

Las trampas en las que caen todos los equipos

Todo responde 200 OK, incluso los errores

El cliente tiene que parsear el body para saber si funcionó. Pierdes el sentido del status code.

Verbos en la URL (GET /borrarProducto)

Repites información que ya está en el método HTTP. Confunde al lector.

Estructura de respuesta inconsistente

A veces array, a veces objeto con data, a veces null. El cliente nunca sabe qué esperar.

Mezclar camelCase y snake_case

Una API con primer_nombre y lastName se siente mal hecha aunque funcione.

Exponer errores internos (stack traces, queries SQL)

Filtros información sensible y abres puertas a atacantes.



Checklist antes de publicar una API

Pasa por todos antes de mandarlo a producción



URLs en plural y minúsculas



Solo verbos HTTP, no verbos en URLs



Códigos de estado correctos (no todo 200)



Estructura de JSON consistente



Errores con código + mensaje útil



Validación de entrada en cada POST/PUT



Paginación en listas potencialmente grandes



Versionado en la URL (/v1)



HTTPS + autenticación



Documentación (Swagger / Postman)

Una API bien diseñada se nota: los clientes la usan sin preguntar nada.



Lo importante en una frase

Una buena API no se nota: hace lo que esperas que haga, responde lo que esperas que responda, y falla de forma predecible.

Convenciones > Inteligencia. Aburrido > Original. Consistente > Perfecto.

Próximo paso: aplicar todo esto al ejercicio de productos